

Technical Report

Multi-Channel Analog Data Acquisition System

ADS1256-Based Datalogger for Strain-Gauge and General-Purpose Analog Sensing

Author: Maryam Moghaddasi

Date: 26 August 2026

1. Introduction

This report documents a low-cost, multi-channel analog data-acquisition (DAQ) system built around the ADS1256 24-bit analog-to-digital converter and an Arduino Uno acting as a USB-to-serial bridge. The system was originally developed to read strain gauges arranged in a Wheatstone bridge configuration, using dummy gauges for bridge completion and per-channel calibration to convert raw ADC readings into engineering units. Because the front end simply digitizes an analog voltage, the same hardware and software chain can log any sensor with an analog output — not strain gauges exclusively.

2. System Overview

The system consists of three layers:

- Analog front end — a Wheatstone bridge completion module (bridge box) that houses the four bridge arms (R1–R4), accepts an excitation voltage on its IN terminals, and produces a differential output on its OUT terminals for each sensor channel.
- Digitizing / acquisition hardware — an ADS1256 breakout board paired with an Arduino Uno. The ADS1256 performs the analog-to-digital conversion for each channel; the Arduino handles channel sequencing and streams the digitized samples to a host computer over USB serial (COM5, 115200 baud).
- Host software — a MATLAB script that opens the serial port, parses incoming samples, applies a per-channel linear calibration, logs the results to a timestamped CSV file, and renders a live plot of all four channels plus their sum.

Up to four independent channels are supported in the current wiring and calibration table, with the terminal blocks on the prototype board (Image 1) providing the $\pm$ connections for each channel and a separate 1–4 channel connector bank below them.

3. Hardware Design

3.1 ADS1256 ADC Module and Arduino Uno

The acquisition core (Image 1, green PCB) is an ADS1256-based ADC breakout board stacked with an Arduino Uno. The ADS1256 is a 24-bit delta-sigma ADC intended for precision, low-level signal measurement such as bridge and thermocouple sensors, which makes it well suited to the small millivolt-level differential signals produced by a Wheatstone bridge. The Arduino Uno reads the digitized values from the ADS1256 and relays them to the host PC over its USB serial connection.

3.2 Wheatstone Bridge Completion Module

Each analog channel is conditioned through a bridge completion box (Image 2) with four resistor-arm positions — R1 (terminals 11/12), R2 (21/22), R3 (31/32), and R4 (41/42) — an OUT pair (1/2) that carries the bridge's differential output to the ADC input, and an IN pair (+/-) that supplies the bridge excitation voltage. A strain gauge (active arm) is wired into one or more of the R1–R4 positions alongside fixed resistors or dummy gauges to complete the bridge, so that strain-induced resistance changes appear as a proportional differential voltage at OUT.

3.3 Wiring and Channel Assignment

On the prototype board, four pairs of screw terminals (+/- each) at the top accept the bridge OUT connections for up to four sensor channels, while a second connector bank below the ADS1256/Arduino stack (labeled 1–4) breaks out the corresponding channel inputs. This terminal layout is what allows dummy gauges or other analog sensors to be swapped in per channel without rewiring the ADC board itself.



Image 1 — ADS1256 / Arduino stack with channel terminal blocks



Image 2 — Wheatstone bridge completion module (R1–R4, OUT, IN)

4. Firmware and Host Software

4.1 Data Framing (Arduino → Host)

Each sample arrives at the host as a comma-separated line terminated with a semicolon, carrying three values: a channel/timing index and the raw ADC reading. The host script derives the active channel from the first value using $\text{channelIndex} = \text{floor}(\text{values}(1)/2) + 1$, which maps the Arduino's index scheme onto four logical channels (1–4).

4.2 MATLAB Host Application

The host-side script (Readingstrainage.m) performs four functions:

- Automatic file versioning — it scans the working folder for existing files named Data_###_<timestamp>.csv, extracts the numeric counter, and starts the next log file at max(counter)+1, so successive runs never overwrite previous data.
- Serial acquisition — it opens COM5 at 115200 baud, reads one line per sample, strips the trailing ';', and splits the line into numeric values.
- Real-time buffering and display — a rolling 100-sample buffer is kept per channel (both raw and calibrated values) plus a fifth buffer holding the sum of all channels; a figure with four per-channel subplots and one combined-sum subplot is updated live via drawnow.
- CSV logging — every sample is written to the log file with the format Time, raw-Channel1, Channel1, raw-Channel2, Channel2, raw-Channel3, Channel3, raw-Channel4, Channel4.

An illustrative excerpt of the CSV structure is shown below (values abbreviated):

Time	raw-Channel1	Channel 1	raw-Channel2	Channel 2	raw-Channel3	Channel 3	raw-Channel4	Channel 4
12:04:31.552	0.00009800	0.997...	-0.00002650	2.582...	-0.00001950	3.406...	-0.00010850	1.151...

5. Calibration Methodology

Each channel is converted from raw ADC counts to engineering units with a linear (single-point-gain, single-point-offset) equation of the form $\text{value} = \text{gain} \cdot (\text{raw} - \text{raw_offset}) + \text{bias}$, applied directly in the acquisition script. The gain (4×10^6 for every channel) and the per-channel raw offset and bias terms — presumably derived from a prior calibration against known reference loads on each bridge — are summarized below:

Channel	Gain	Raw Offset	Bias / Zero Offset	Conversion Equation
1	4×10^6	+0.000094	0.61	$\text{value} = 4e6 \cdot (\text{raw} + 0.000094) + 0.61$
2	4×10^6	-0.000027	2.5926	$\text{value} = 4e6 \cdot (\text{raw} - 0.000027) + 2.5926$
3	4×10^6	-0.000020	3.4076	$\text{value} = 4e6 \cdot (\text{raw} - 0.000020) + 3.4076$
4	4×10^6	-0.000109	1.1538	$\text{value} = 4e6 \cdot (\text{raw} - 0.000109) + 1.1538$

Because the gain and offsets are hard-coded per channel rather than computed at runtime, any change in bridge wiring, excitation voltage, or gauge factor requires re-deriving and updating these four constants before the logged values can be trusted as engineering units.

6. System Capabilities

- Reads up to four analog channels simultaneously with live plotting and CSV logging.
- Bridge-based front end supports strain gauges (quarter-, half-, or full-bridge, using dummy gauges for completion) but is not limited to them — any sensor with a compatible analog output can be connected to the same ADS1256 inputs.

- Automatic, non-destructive file naming means repeated test runs are preserved as separate, sequentially numbered datasets.
- The rolling-buffer display gives immediate visual feedback per channel and on the combined signal sum, useful for spotting bridge imbalance or noisy channels during a test.

7. Conclusion and Future Work

The system demonstrates a functional, low-cost multi-channel strain/analog datalogger built from an ADS1256 ADC, an Arduino Uno, and a MATLAB host application, with bridge-completion hardware that makes it straightforward to attach strain gauges via dummy-gauge bridges. Suggested next steps for a fuller report or a production version include: documenting the Arduino firmware alongside the host script, recording the excitation voltage and gauge factor used for the calibration in Section 5, adding sample-rate and noise-floor figures for the ADS1256 front end, and validating the linear calibration against a set of known reference loads.